Cross-ISA prediction: the probe transfers within an architecture, not across one

The decisive experiment. dev = Apple M1 (arm64). prod = **eight** machines: seven Docker cells on the same arm64 host, and one **x86_64 AMD EPYC 9V74** measured on a GitHub Actions runner via `.github/workflows/x86-operator-probe.yml`. Every machine probed at reps=9, every record passing the repeatability gate, and dev and all prod cells verified to agree on `work_mem`, `shared_buffers`, `effective_cache_size`, `random_page_cost`, `max_parallel_workers_per_gather` and concurrency. 22 TPC-H queries at SF1, 176 pairs.

Replicated across four independent x86 machines

prod	CPU	naive	operator, zero-shot
gha-x86	AMD EPYC 7763	21.41%	**72.84%**
gha-ubuntu-latest	AMD EPYC 7763	21.00%	**69.44%**
gha-ubuntu-24.04	AMD EPYC 7763	21.79%	**66.15%**
gha-ubuntu-22.04	AMD EPYC 7763	22.75%	**67.26%**
mean		*21.7%*	***68.9%***

Four separate GitHub Actions jobs, each on a fresh physical host. The failure reproduces on **every one**: the model is consistently ~3.2× worse than assuming production equals development. The four signatures agree closely with each other (seqscan 7.13–7.46M tuples/s, parallel speedup 2.160–2.167), so this is not probe noise — it is a systematic property of predicting across the architecture boundary.

A fifth measurement on a **different** x86 CPU (AMD EPYC 9V74, from the first workflow run) gives seqscan 8.75M and speedup 2.211 — 18% apart from the 7763 cluster on the serial axes, which is a reminder that "x86" is not one machine either.

The result

population	naive (prod = dev)	operator model, zero-shot
same ISA (7 arm64 cells, 154 pairs)	37.39%	**29.31%**
cross ISA (4 x86_64 machines, 88 pairs)	**21.7%**	**68.9%**
all 11 production machines, 242 pairs	31.70%	43.72%

Within an architecture the model works. Across one it is 3.4× worse than doing nothing.

Per machine, with few-shot calibration scored only on unrevealed queries:

prod	ISA	naive	k=0	k=3	k=5	scalar floor
c1m1g	arm64	62.58	**23.47**	27.94	24.65	30.43
c1m3g	arm64	59.78	**31.26**	27.31	26.12	31.33
c2m3g	arm64	22.17	22.15	28.19	25.06	16.97
c4m1g	arm64	32.41	36.00	29.34	23.91	17.68
c4m3g	arm64	32.70	36.40	25.84	26.27	17.03
c8m1g	arm64	30.84	33.42	24.30	24.30	15.04
c8m3g	arm64	21.27	22.49	29.96	28.21	22.33
gha-x86	**x86_64**	21.41	**72.84**	**23.86**	**23.38**	16.75

The mechanism, and it is not what was predicted

The probe reports the M1 faster on **every** axis, by 1.09–1.49×, with a higher parallel speedup (2.625 against 2.211). The model therefore predicts the x86 machine is **1.667× slower**. It is actually **1.044×** — essentially the same speed. A 60% over-statement of the capability gap, which is why a 21% problem becomes a 73% one.

It is not operator-mix divergence. That hypothesis was tested directly (cross_isa_operator_rates.md) and refuted: the arm64 → x86_64 operator-ratio spread is **1.365×**, while the worst pair *within* the arm64 grid is **1.371×**. The ISA boundary perturbs relative operator costs no more than a CPU quota change on one machine.

What fails is the **absolute scale**. Tuples-per-second on a synthetic operator is not comparable across microarchitectures: the same SQL does different amounts of real work per tuple on different cores, in ways that do not predict query throughput. This is the same failure as the original FP-GFLOPS signature (324% MAPE laptop → EC2) — a different proxy, the same defect. **Replacing one architecture-sensitive scalar with five did not fix it.**

Calibration is not an optimisation; across an ISA it is the fix

Five queries measured on production take the x86 cell from **72.84% to 23.38%**, against a per-machine-scalar floor of 16.75% for that pair. The information needed was always present — the probe simply had the wrong absolute scale, and one multiplicative correction removes any constant error exactly. This is the same effect already seen on EC2, where calibration made the `nproc`-versus-DOP definition bug (337.75% against 160.68% zero-shot) **completely unidentifiable**.

Note the honest limit: calibrated, the cross-ISA model reaches **23.38%** against naive's **21.41%**. It achieves *parity with assuming production equals development* — it does not beat it. On this pair, the capability model contributes nothing that calibration did not.

What this means for the method

1. **Capability probing transfers within an ISA and not across one.** Same-ISA, the model earns 8 points over naive (29.31% against 37.39%). Cross-ISA it earns nothing and, left uncalibrated, is actively dangerous.
2. **Any deployment crossing architectures must calibrate.** Uncalibrated zero-shot cross-ISA prediction is the one configuration measured to be far worse than doing nothing, three times now (EC2 at 324%/160%, this at 72.84%).
3. **The remaining headroom is not in the capability signature.** Per-operator rates add –0.08 pp over a single scalar given the same parallelism information, and the ISA experiment shows the operator mix barely moves. What is left is per-query variance that no per-machine quantity can express, and the storage regime, for which no probe axis exists.

Scope

- **Four x86 machines, but only two distinct CPU models**, both AMD EPYC, both on Azure-hosted GitHub runners. The failure replicates 4/4 and the signatures are tight, so the finding is not a fluke — but a non-AMD x86 machine (Intel) and a non-CI host would both strengthen it, and neither has been measured.
- A CI runner is shared hardware; the record passed the repeatability gate but neighbours are not controlled.
- 22 queries, SF1, single client, identical data on both sides, one engine version.
- `k=3` / `k=5` columns average 40 random reveals per cell; `spread` selection is reported in `results/laptop_to_prod_metrics.json`.